

HeritageHub

Group 6: Emily Youngward, Emily Feng, Joshua Chin

IAT 459 Spring 2026

Application Overview

HeritageHub is a web application where anyone interested in Canadian heritage can explore and review museums, galleries, and historic sites across Canada. HeritageHub is community focused through its user contributions to the database. Users can contribute new site additions, reviews with images, and to their own private journals/lists. HeritageHub was built using MongoDB, Node.js, Express.js, and React. The application features JSON Web Token based authentication along with the React Leaflet API, and Multer for image uploading. The application's code also makes use of bcrypt, Context API, and JWT decode.

Technical Architecture

API Documentation Table:

<i>Method</i>	<i>Endpoint</i>	<i>Description</i>	<i>Auth Required?</i>
POST	/api/auth/register	Register new user and hash password	No
POST	/api/auth/login	Authenticate user and return JWT	No
GET	/api/facility	Retrieve all facilities (newest first)	No
GET	/api/facility/my-facilities	Retrieve facilities for the logged in user	Yes (Member or Admin)
GET	/api/facility/:id	Retrieve facility by id	No
POST	/api/facility	Create a new facility entry	Yes (Member or Admin)
DELETE	/api/facility/:id	Delete your facility, or any facility if admin	Yes (Member or Admin)
PUT	/api/facility/:id	Edit your facility, or any facility if admin	Yes (Member or Admin)
GET	/api/reviews/:facility	Retrieve reviews for a specific facility	No
GET	/api/reviews	Retrieve reviews from all facilities	Yes (Admin only)
POST	/api/reviews	Create a new review for a facility	Yes (Member or Admin)
DELETE	/api/reviews/:id	Delete your review, or any review if admin	Yes (Member or Admin)
GET	/api/users/me	Get user data and stats for bucket list/visited	Yes (Member or Admin)
GET	/api/users	Retrieve all users	Yes (Admin only)
POST	/api/users/bucket/:facilityId	Add a facility to your bucket list	Yes (Member or Admin)
POST	/api/users/visited/:facilityId	Add a facility to your visited list	Yes (Member or Admin)
DELETE	/api/users/bucket/:facilityId	Delete a facility from your bucket list	Yes (Member or Admin)
DELETE	/api/users/visited/:facilityId	Delete a facility from your visited list	Yes (Member or Admin)
DELETE	/api/users/:id	Delete a user	Yes (Admin only)
PUT	/api/users/:id/role	Change a users role to member or admin	Yes (Admin only)

Finalized Schema:

FACILITY		REVIEWS		USER	
Name	(type: String)	facility	(type: ObjectId)	username	(type: String)
Category	(type: String)	rating	(type: Number)	email	(type: String)
Province	(type: String)	comment	(type: String)	password	(type: String)
City	(type: String)	image	(type: String)	role	(type: String)
Address	(type: String)	user	(type: ObjectId)	visited	(type: Array)
Latitude	(type: Number)			bucketList	(type: Array)
Longitude	(type: Number)				
PostalCode	(type: String)				
LastReviewImage	(type: String)				

Role Logic:**Visitors:**

- Are able to *Browse & Explore* all the museums, galleries, and cultural sites listed on the home page when you enter the website.
- Can use the *Search & Filter* option on the home page.
- They are unable to *Add* new places or *Delete* anything on the website.
- Can view details for each facility (including reviews) and see the Map.js page.

Members:

- After registering they are given a (JWT Token) once logged in.
- Any facilities created by the logged in user are associated with their account, this is the same for reviews.
- Same function as Visitor once they visit the home page in ADDITION:
 - Able to add new museums and cultural sites they discover to our database.
 - Access the manage facilities dashboard to manage facility contributions.
 - Have their Username displayed on the website and a welcome message included.
 - View a personal Bucket List page (places you wish to go) and Travel Journal page (places you have been).
 - Add a review, and add the facility to their personal Bucket List or Travel Journal.

Admins:

- Has all the Member permissions, in ADDITION:
 - Can delete or remove any information, including other users' content for the facilities on the website (Just in case info is outdated or incorrect)
- Has final say as to what appears on the website
- Manually assigned to database

Execution Report

Completion Status:

Main features of our initial proposal

Feature	Visitor	Member	Admin
Browse Database & Map	View Only	View Only	View Only
Filter by Province/Tag	Yes	Yes	Yes
View Reviews/Ratings	Yes	Yes	Yes
Create Reviews/Ratings	Yes	Yes	Yes
Access Travel Journal	No	Personalized	Personalized
Access Bucket List	No	Personalized	Personalized
Delete Others' Content	No	No	Yes

All main features were implemented. Some additional secondary features that we did not implement due to time constraints include:

- Password Resetting - also was not covered in class.
- Map marker clustering for large dataset
- Plot location on map instead of entering actual longitude/latitude values
- Extensive facility "tags" system

Technical Challenges:

- We had issues implementing image uploading and saving at first. We were not quite sure how to upload the images to MongoDB so they would show on our application. The images were uploaded to MongoDB but in an unreadable format. After some further research we then learned that we needed to convert the images into a Base64 string. This ultimately solved our problem.

```
// POST ROUTE (create review - protected for only logged in users)
router.post("/", verifyToken, upload.single("image"), async (req, res) => {
  try {
    let imageBase64 = null;

    //if an image file was uploaded, convert it to base64 for MongoDB storage
    if (req.file) {
      const mimeType = req.file.mimetype; //get mime type (ex. png, jpeg)
      const base64Data = req.file.buffer.toString("base64"); //convert buffer to base 64 string
      imageBase64 = `data:${mimeType};base64,${base64Data}`; //format as data URI
    }
  }
});
```

- We also had an issue where if a facility was deleted but it happened to be in the user's Bucket List or Travel Journal, the application would crash. To fix this issue we went into the facility.js route and updated the delete route. We added User.updateMany to the delete route, this updates every user document and removes the facility from their bucket lists and travel journals. This solved our crashing issue.

```

await User.updateMany(
  {},
  {
    $pull: {
      bucketList: req.params.id,
      visited: { facility: req.params.id },
    },
  },
);

```

Teamwork

Work Distribution Table:

Name	Contribution
Emily Youngward	• Set up the Express server and established the connection to MongoDB Atlas. • Designed and implemented all Mongoose data schemas (User, Facility, Reviews). • Developed the core API routes for Authentication, Facilities, and Reviews. • Implemented the JWT authentication flow and the verifyToken authorization middleware. • Created the seed.js script to populate the database from the provided CSV file. • Created all the functioning features of BucketList.js, TravelJournal.js, AddReview.js, and FacilityDetails.js. Including image uploading for reviews. • Added review averaging and review image displays on facility cards. Also created pagination and filtering in the backend for Home.js.
Joshua Chin	• Developed Home.js, Dashboard.js, AdminDashboard.js, Login.js, Register.js, and Map.js. • Implemented client-side data fetching (useEffect, fetch) to communicate with the backend API. • Integrated the AuthContext to manage global authentication state (login, logout, token storage). • Created the ProtectedRoute wrapper component to secure client-side routes. • Implemented roles, including backend and frontend features for admin role. • Implemented map API for Map.js and FacilityDetails.js

Emily Feng	• Refined the overall user interface and user experience with detailed CSS styling. • Developed and iterated on the main HeritageHub.css • Styled the authentication pages (Auth.css) to match the visual theme of the main site. • Enhanced the visual design of the facility cards, navigation bar, and hero section.
-------------------	---

Individual Reflections:

Emily Youngward: Through this project I feel I have learned effectively how to implement fullstack development methods including React in the frontend and Node.js and Express in the backend. I also now know how to implement a NoSQL database with MongoDB. I can now create well structured schemas that have meaningful impacts in the application. I also strengthened my skills in user interface design. For example, when building HeritageHub we had to constantly consider what problems a user might have when navigating our application. Overall, I now feel more confident in my ability to create a well thought out web application that considers not only user interaction, but also security and data storage.

Joshua Chin: This project helped to further strengthen my problem-solving skills, especially as React was new for me. Communication between frontend and backend is an aspect of project work that required me to keep up with understanding the environment around how data is requested and received across the site. When implementing routing, I also learned more about security and accounting for how users may choose to navigate, including unexpected or unintended actions. This helped me to think about a wider range of scenarios and was validated through peer reviews. They reinforced features that were accomplished successfully and showed additional problems I hadn't considered initially, such as admin features built for any possible user scenario to avoid crashes, like adding optional images or deleting content in different spots.

Emily Feng: Working on this project helped me strengthen my skills in connecting design with actual working code, which was something I hadn't done much of before. Matching the React app to our Figma prototype taught me that building a UI isn't just about making it look right but small details like where star ratings appear or whether images load correctly can meaningfully affect how usable the app feels for someone trying to get things done quickly. Working on the map performance was where I learned the most. I had to think through how a real user would interact with the page by implementing the zoom threshold and search logic made me more aware of how frontend decisions directly impact what gets requested from the backend, and why it matters to be intentional about when and how that data is fetched. I also got better at thinking through edge cases like what happens when a user searches before zooming in, or when province data is stored differently than how someone would naturally type it.